Guía de instalación y configuración de ExpoTec

Frontend React Native y backend Express

Versión de referencia 0.4.0

25 de octubre de 2026

Contenido

Alcance 3

Requisitos 3

Obtener y ubicar el código 3

Instalar las dependencias 3

Configurar el backend 4

Preparar TiDB y aplicar las migraciones 5

Iniciar la API 5

Ejecutar la web local con sesión 6

Crear el primer administrador 6

Ejecutar o instalar Android 6

Configurar funciones externas 7

Verificar la instalación 8

Despliegue y mantenimiento 8

Resolver problemas de configuración 9

Documentación técnica de consulta 9

Alcance

Esta guía permite instalar las dependencias, configurar la base de datos y ejecutar ExpoTec para desarrollo. También explica cómo probar la APK y cómo se separan los despliegues. El código activo del frontend está en `frontend/native`; `frontend/web` conserva la referencia anterior y no es el cliente que se instala con esta guía.

La aplicación publicada se consulta en ExpoTec web. Su API está en ExpoTec API. Para usar la aplicación publicada o instalar la APK no hace falta preparar un entorno de desarrollo.

Requisitos

Para desarrollar, necesitás Node.js 24.3 o posterior dentro de la serie 24, npm, Git y acceso a una base TiDB. El backend declara Node 24.x. Las instrucciones de comandos están escritas para PowerShell. Para Android necesitás un teléfono compatible o un emulador configurado; la APK actual requiere Android 7 o posterior.

Para configurar las funciones externas necesitás aplicaciones OAuth de Google, Discord y GitHub, almacenamiento privado de Vercel Blob para fotos y un servicio SMTP para recuperación. Estas funciones son opcionales en una instalación propia: sus botones o acciones dependen de la configuración disponible.

El proyecto utiliza el controlador `mysql2` y migraciones SQL en TiDB. Una instalación sobre otro servidor MySQL debe verificar compatibilidad de migraciones y transacciones; no se afirma que ese entorno alternativo haya sido probado.

Obtener y ubicar el código

Cloná el repositorio y entrá en la carpeta de ExpoTec:

```
git clone https://github.com/NachoTorress/PDeISC.git
cd PDeISC
cd "Proyecto Final - ExpoTec"
```

Si ya tenés el repositorio, usá tu copia existente. La raíz Git contiene otros proyectos. Los comandos siguientes parten de la carpeta de ExpoTec, salvo que se indique un subdirectorio.

La estructura principal es `frontend/native` para la aplicación, `backend` para la API, `backend/sql` para las migraciones y `docs` para la documentación. Las carpetas de dependencias y los archivos privados de entorno no forman parte de los entregables.

Instalar las dependencias

En una terminal, prepará el backend:

```
cd backend
npm ci
```

En otra terminal, desde la carpeta de ExpoTec, prepará el frontend:

```
cd frontend/native
npm ci
```

`npm ci` usa los archivos de bloqueo incluidos en el proyecto. Conservá ambas terminales para ejecutar los servicios por separado.

Configurar el backend

En `backend`, copiá los ejemplos y completá los valores privados con tu configuración:

```
Copy-Item .env.example .env
Copy-Item .env.setup.example .env.setup
```

Si esos archivos ya existen, revisalos sin sobrescribir las credenciales guardadas. `.env` contiene la configuración de ejecución. `.env.setup` permite usar credenciales administrativas únicamente en las tareas de preparación y migración; sus valores prevalecen en esos comandos.

Variables esenciales

Variable	Uso
NODE_ENV y PORT	Entorno local y puerto de la API; para desarrollo, development y 3001.
CORS_ORIGINS	Orígenes exactos del frontend separados por comas, sin barra final.
JWT_SECRET	Secreto aleatorio de al menos 32 bytes para firmar los tokens.
DB_HOST y DB_PORT	Servidor TiDB y puerto; el valor predeterminado es 4000.
DB_USER y DB_PASSWORD	Credenciales del usuario de base de datos.
DB_NAME	Base de la aplicación; la instalación prevista utiliza expotec.
DB_SSL y DB_SSL_CA	TLS y certificado CA opcional. Producción exige TLS.
DB_POOL_LIMIT	Límite de conexiones; el valor predeterminado es 5.

Generá un secreto propio y guardalo solo en tu configuración privada:

```
node -e "console.log(require('node:crypto').randomBytes(48).toString('base64url'))"
```

En el entorno local de esta guía, autorizá `http://127.0.0.1:5173` en CORS_ORIGINS. Si usás otro origen, agregalo de forma exacta. No coloques credenciales de TiDB, secretos OAuth, JWT ni SMTP en el frontend ni en variables EXPO_PUBLIC_.

Preparar TiDB y aplicar las migraciones

Creá una base vacía `expotec` en tu instancia propia de TiDB y un usuario de aplicación con permisos SELECT, INSERT, UPDATE y DELETE sobre ella. Completá `.env` con esas credenciales. En `.env.setup`, configurá una cuenta con permisos para crear y modificar tablas dentro de la misma base.

Desde `backend`, aplicá el esquema y comprobá la conexión:

```
npm run db:migrate
npm run db:check
```

El migrador registra las migraciones aplicadas y comprueba sus sumas de verificación. No edites archivos SQL ya aplicados: los cambios nuevos requieren otra migración. La serie actual va de `001_auth.sql` a `010_password_recovery.sql` y cubre cuentas, sesiones, espacios, proyectos, inventario, préstamos, galería, accesos sociales, fotos y recuperación.

El script `db:setup` automatiza una preparación específica de TiDB: exige DB_NAME=expotec, un TEST_DB_NAME con prefijo expotec_test_ y un nombre de usuario con el formato que valida el script. Para una instalación manual como la anterior, no es necesario ejecutarlo. No lo uses sobre una base ajena ni adaptes credenciales de producción para pasar sus validaciones.

Las migraciones no cargan los proyectos ni los planos ficticios de la demostración. Los datos reales se crean desde las pantallas autorizadas de la aplicación.

Iniciar la API

Desde `backend`, ejecutá:

```
npm run dev
```

Para una ejecución sin recarga automática, usá `npm start`. La API local escucha en el puerto 3001 salvo configuración distinta. Comprobá salud y conexión a la base:

```
Invoke-RestMethod -Method Post `
  -Uri 'http://127.0.0.1:3001/api/health' `
  -ContentType 'application/json' -Body '{}'
Invoke-RestMethod -Method Post `
  -Uri 'http://127.0.0.1:3001/api/health/ready' `
  -ContentType 'application/json' -Body '{}'
```

La segunda consulta debe responder correctamente e indicar `database: connected`. Una respuesta saludable no prueba por sí sola un recorrido de registro o préstamo; esos casos se verifican desde la aplicación o con las integraciones de pruebas.

Ejecutar la web local con sesión

Para comprobar la sesión web con el proxy incluido, abrí la terminal del frontend y dejá sin definir EXPO_PUBLIC_API_URL, tanto en la sesión como en sus archivos `.env`. El valor del ejemplo apunta directamente a la API y no corresponde a este recorrido mediante proxy.

```
Remove-Item Env:EXPO_PUBLIC_API_URL -ErrorAction SilentlyContinue
$env:EXPO_NO_DOTENV = '1'
npm run export:web
$env:PORT = '5173'
$env:API_PROXY_URL = 'http://127.0.0.1:3001'
npm run preview
```

Abrí `http://127.0.0.1:5173`. El servidor de vista previa reenvía `/api` al backend local y mantiene frontend y API bajo el mismo origen del navegador. Debe coincidir con CORS_ORIGINS. Es una exportación estática: después de cambiar el código, volvé a exportar para actualizarla.

Para trabajar con recarga de Expo podés usar `npm run web`; la URL aparece en la terminal. La configuración de sesión y los proveedores deben corresponder a ese origen. Para una comprobación reproducible del login web se recomienda el recorrido con proxy anterior. No abras un archivo index.html directamente.

Crear el primer administrador

Desde la aplicación conectada a tu API local, registrá una cuenta propia. Luego, desde `backend`, asignale el permiso inicial:

```
npm run admin:grant -- correo@escuela.edu.ar
```

Reemplazá el correo de ejemplo por el de la cuenta que acabás de registrar. El comando utiliza la conexión de `.env` y requiere permisos sobre las tablas involucradas. No crea una cuenta ni una contraseña predeterminada. Después actualizá los permisos de la sesión o volvé a ingresar.

El administrador puede gestionar usuarios, crear espacios, habilitar mesas como puestos y cargar inventario. Los profesores y encargados se registran primero y reciben sus permisos desde **Usuarios y permisos**. La demostración no sustituye esta preparación de datos compartidos.

Ejecutar o instalar Android

Instalar la APK disponible

Descargá ExpoTec 0.4.0 para Android desde el teléfono, abrí el archivo e instalalo. El paquete es `ar.edu.expotec.organizacion`, con versionCode 5. La APK se conecta a la API publicada y funciona sin un servidor de desarrollo local.

El usuario confirmó el 25 de octubre de 2026 que la APK funciona bien en su uso y que el ingreso nativo con Google funciona. Esa confirmación no se extiende automáticamente a todos los teléfonos ni a recorridos no mencionados.

Trabajar con el código nativo

`npm start` inicia Expo y `npm run android` intenta abrir el entorno Android disponible. El proyecto usa módulos nativos, incluido el acceso de Google; Expo Go no reemplaza una compilación que incluya esos módulos. Para comprobar esas funciones usá la APK o una compilación de desarrollo compatible.

Sin EXPO_PUBLIC_API_URL, Android usa la API publicada. Para apuntar desde el emulador estándar a una API en la computadora, el cliente admite `http://10.0.2.2:3001`. La validación actual no admite una dirección LAN HTTP arbitraria para un teléfono físico; para ese caso usá un origen HTTPS accesible. Una compilación incorpora la configuración disponible al construirla: cambiar una variable en la computadora no modifica una APK ya instalada.

Generar otra APK

Con una cuenta autorizada en el proyecto de Expo, desde `frontend/native`:

```
$env:EAS_NO_VCS = '1'
$env:EAS_PROJECT_ROOT = (Get-Location).Path
$env:EXPO_NO_DOTENV = '1'
npx eas-cli build --platform android --profile preview
```

El perfil preview genera APK; el perfil production no debe confundirse con él, porque su salida es un Android App Bundle. Antes de distribuir otra versión, revisá firma, número de versión, contenido del archivo enviado e instalación. La guía técnica `docs/APK_PRUEBA.md` conserva los detalles de compilación y verificaciones.

Configurar funciones externas

Google Discord y GitHub

Configurá en el backend el par SOCIAL_GOOGLE_CLIENT_ID y SOCIAL_GOOGLE_CLIENT_SECRET; repetí el esquema para Discord y GitHub. SOCIAL_WEB_ORIGIN debe ser el origen autorizado del frontend y pertenecer a CORS_ORIGINS. Un proveedor sin credenciales no se ofrece; un par incompleto se rechaza.

Los retornos OAuth web pasan por `/api/auth/social/google/callback`, `/api/auth/social/discord/callback` o `/api/auth/social/github/callback` bajo el dominio del frontend. Para una instalación propia, registrá sus direcciones exactas en cada proveedor. La configuración Android de Google requiere además el paquete y certificado de firma correspondientes. Consultá `docs/ACCESOS_SOCIALES.md` y `docs/ANDROID_GOOGLE.md` para los pasos específicos; no copies los secretos de otra instalación.

Fotos

Conectá un almacén privado de Vercel Blob al backend y configurá BLOB_READ_WRITE_TOKEN solo allí. La API prepara permisos temporales de carga, valida el contenido y controla su publicación. La carga real usa JPG, PNG o WebP de hasta 10 MB. Consultá `docs/CARGA_FOTOS.md` para límites, validación y transporte nativo y web.

Recuperación de contraseña

Completá SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD y PASSWORD_RESET_WEB_ORIGIN. Para Gmail se utiliza una contraseña de aplicación. El origen de recuperación debe estar autorizado en CORS_ORIGINS. La disponibilidad se consulta en POST `/api/auth/recovery/query`; para acreditar el envío hay que recibir y usar un correo real de una cuenta propia. Los detalles están en `docs/RECUPERACION_CONTRASENA.md`.

Verificar la instalación

En `backend`, ejecutá `npm test`. En `frontend/native`, ejecutá las comprobaciones:

```
npm run typecheck
npm run lint
npm test
npm run export:android
npm run export:web
```

La exportación Android verifica el empaquetado JavaScript y los recursos; no crea una APK. Las pruebas locales tampoco sustituyen probar el dispositivo ni autorizar los tres proveedores reales.

Las integraciones del backend tienen scripts como test:spaces, test:projects, test:inventory y test:loans. Requieren una base exclusiva con prefijo `expotec_test_` distinta de producción. La prueba test:database exige además una base vacía. Prepará ese entorno según `backend/README.md`; no ejecutes integraciones sobre la base real.

Despliegue y mantenimiento

Frontend y backend se despliegan por separado en Vercel desde main. La raíz del frontend es `Proyecto Final - ExpoTec/frontend/native` y la del backend, `Proyecto Final - ExpoTec/backend`. El frontend exporta la web y utiliza su reescritura `/api`; el backend conserva las variables privadas y conecta con TiDB usando TLS.

Para una instalación con dominios diferentes, actualizá las reescrituras del frontend, la API de las compilaciones Android, CORS_ORIGINS y los orígenes y retornos de proveedores y recuperación. No basta con cambiar el nombre del proyecto en Vercel. Aplicá las migraciones antes de publicar código que las necesite. Conservá copias de seguridad de la base y ejecutá la limpieza `npm run db:prune` como parte del mantenimiento previsto.

Resolver problemas de configuración

Problema	Revisión
La API no inicia	Variables obligatorias, secreto JWT, pares OAuth completos y conexión a TiDB.
La base no conecta	Host, puerto, usuario completo, contraseña, TLS, permisos y red.
El login web no conserva sesión	Mismo origen mediante proxy, CORS exacto y ausencia de EXPO_PUBLIC_API_URL en la exportación con proxy.
El emulador no llega a localhost	Usar 10.0.2.2 para la computadora anfitriona en el emulador estándar.
No aparecen botones sociales	Credenciales de cada proveedor y origen configurados en el backend.
Google falla en otra APK	Paquete, certificado de firma y configuración Android del proveedor.
El inventario está vacío	Cargar datos como administrador; la demo no importa sus existencias a la base.
La vista previa muestra código anterior	Detener la vista previa, volver a exportar y reiniciarla.

Documentación técnica de consulta

El contrato de la API se mantiene en `backend/README.md`, con rutas, métodos, cuerpos, respuestas y códigos de error. La API actual utiliza principalmente POST para sus operaciones JSON y tiene excepciones GET y PATCH documentadas. Esta guía describe ese contrato existente; no implica una migración a otros métodos HTTP.

Para profundizar: `docs/DESPLIEGUE.md`, `docs/APK_PRUEBA.md`, `docs/ACCESOS_SOCIALES.md`, `docs/ANDROID_GOOGLE.md`, `docs/CARGA_FOTOS.md` y `docs/RECUPERACION_CONTRASENA.md`. El manual de usuario explica las tareas y la guía de demostración organiza la presentación de los perfiles.